CS6005: Bujet

Zachary Beck

dyson
institute of
engineering+
technology

1. Project Ideation [20%]

1.1 Project ideation and overview

System Name: Bujet

Bujet is an all-in-one budgeting app that reduces impulse spending and provides spending insights by combining the user's transaction data. User's can securely connect to multiple bank accounts and credit cards to aggregate transaction data and set categorical budgets. Bujet uses the transaction data to display whether the user is within budget and incorporates a unique 'impulse spend' checker. Instead of looking into past transactions, a user can add add a transaction and check if it exceeds their budget or not with a helpful AI paragraph, grounded in pre-calculated values, explaining the best option out of remind me later, set as a wish-list or buy now. Bujet also includes a helpful 'saved by pausing' amount, to reinforce the savings made possible by not impulse spending.

Major features:

- Connects to Bank Accounts through TrueLayer using OAuth meaning Bujet never sees or handles your sensitive bank account credentials.
- Aggregation of transaction data.
- Users can set budgets for categories set out in the app.
- Impulse spend checker.
- Manual transaction import on top of automated transaction import.
- A monthly money saved by not impulse spending amount.

Operating environment:

a. Available as an iOS app
b. Required HW:
    a. Smartphone capable of running iOS apps and internet connectivity.
c. Required SW:
    a. Using Open Banking technology to connect to bank accounts and credit card.

Pros:

a. Open Banking connection means Bujet doesn't handle your bank account credentials.
b. Reduction in impulse spending.
c. Aggregation of transaction data across bank accounts and credit cards.

Cons:

a. Certain bank accounts don't have OAuth enabled, so bank account credentials will pass through TrueLayer servers.
b. User must actively use the impulse spend checker within the app, takes conscious effort.
c. Categories might not apply well to all transactions, banks and credit card issuers interpret categories differently.

1. Project Ideation [20%] (cont'd)

1.2 Analysis of similar systems

System Name: Snoop

Snoop is a UK-focused budgeting app that connects to user's bank accounts (without asking for their password) through Open Banking technology to aggregate multiple bank accounts so user's can see their spending and budgeting objectives all in one place. It can categorise spending automatically, enabling users to understand what they're spending the most money on and how this changes each month.

Snoop's main highlight is its focus on money-saving suggestions ("Snoops") on places the user already spends, such as special offers to reduce cost, and highlights areas that need the user's attention, such as bill increases and warns if the user's account won't cover the bills.

Major features:

- Connects to Bank Accounts through Open Banking through the bank itself, meaning Snoop never handles your bank account credentials.
- Automatic spending categorisation and analysis, such as spending this month compared to previous month.
- Budgets can be set quickly, advertised in as little as two presses, with automatic budget setting by Snoop as well.

Operating environment:

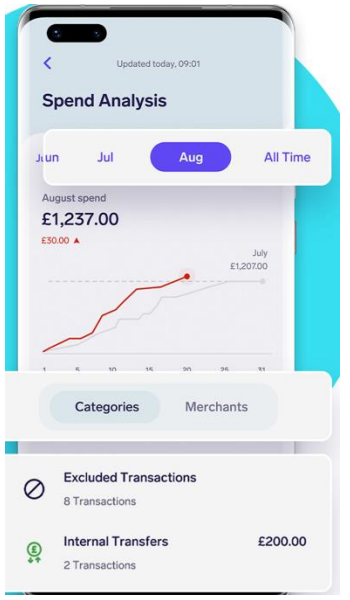
a. Available as iOS and Android apps
b. Required HW:
    a. Smartphone capable of running iOS and Android apps and internet connectivity.
c. Required SW:
    a. Using Open Banking technology to connect to bank accounts and credit cards, needs to be renewed every 90 days.

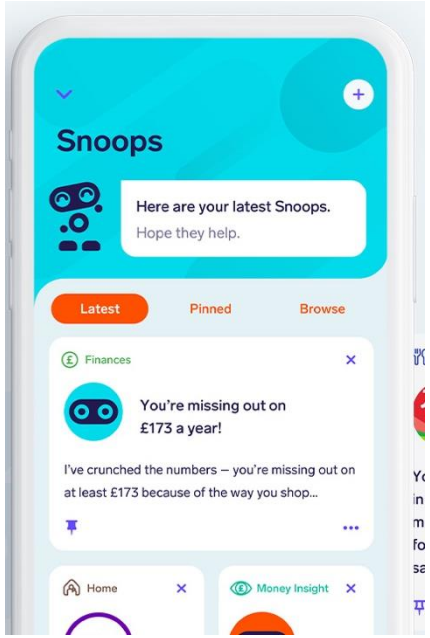
Pros:

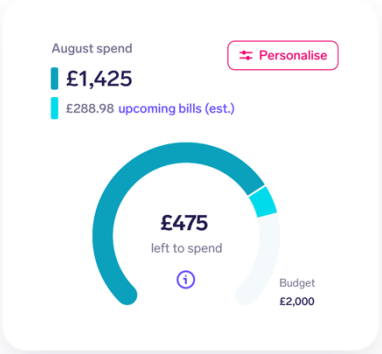
a. Open Banking connection means Snoop doesn't handle your bank account credentials.
b. Strong "all in one place" experience, multiple credit cards and bank accounts can be linked together.
c. Clear money saving proposition because of active bill monitoring and providing context-based suggestions to save money.
d. Daily balance notifications and weekly/monthly summaries keep users aware without having to login to the app themselves.

Cons:

a. Open Banking reconnection friction, need to reconnect every 90 days to maintain experience.
b. Lots of good features are hidden behind a pay wall (Snoop Plus) such as tracking total net worth, creating unlimited spending alerts and tracking bills by pay cycle.
c. Retrospective only, no way to check before you spend only after.





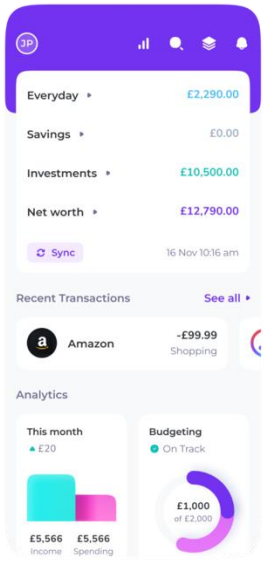


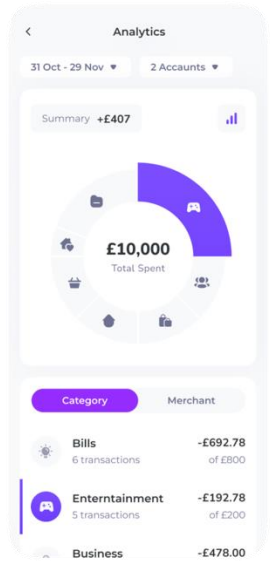
1. Project Ideation [20%] (cont'd)

1.2 Analysis of similar systems

System Name: Emma

Emma is a personal finance and budgeting app that, similarly to Snoop, uses Open Banking to aggregate bank accounts, credit cards and investments into one place. It is used to set budgets, identify wasted spending and monitor overall financial health. It has both free and paid tiers.



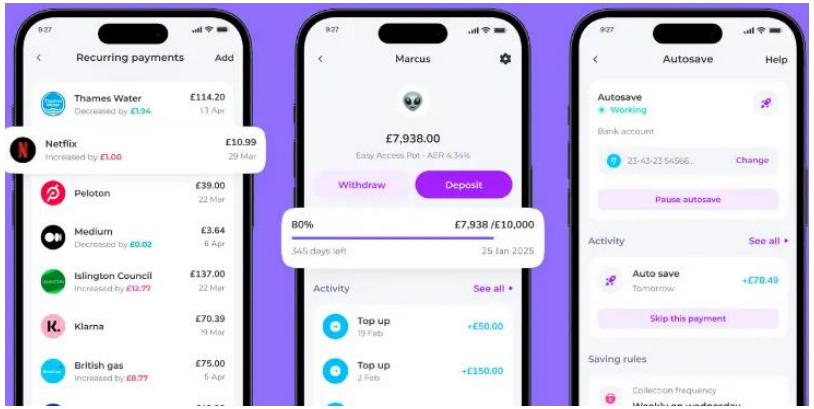


Emma's main highlight is their "true balance" technology to work out how much money you have left after recurring payments are taken. It also has cashback rewards and investment options for US stocks. User's can also make custom categories in the paid tiers.

Major Features:

- Offline accounts that aren't related to any sort of bank account.
- Account aggregation, overall view of multiple accounts together.
- Automated budgeting with custom categories (or automated categories)
- Subscription tracking to identify waste.
- Weekly/monthly spending summaries

a. Operating Environment:
    a. Platform:
        a. Android and iOS apps
    b. HW:
        a. Smartphone with internet connectivity capable of running apps with biometric login capability is preferable.
    c. SW:
        a. Relies on network connectivity to add bank account and maintain syncing for transaction data.



Pros:

a. Emphasis on tracking subscriptions and bills, warning users through notifications.
b. Lots of different subscription plans (free, plus, pro, ultimate) to tailor use case and provide more features to those who require it.
c. Markets themselves as having "bank-grade security" utilising Open Banking to facilitate logins.

Cons:

a. Free tier has significant limitations, such as limiting number of connected accounts, transaction history is limited and no advanced budgeting tools (subscription killer and custom spending categories).
b. High subscription cost, with £15/month for the highest tier.
c. Only retrospective spending, nothing to check a future transaction.

1. Project Ideation [20%] (cont'd)

1.2 Analysis of similar systems

System Name: YNAB (You Need A Budget)

YNAB is built around the principle of zero-based budgeting, where every pound is given a job by assigning money to specific categories before it is spent.

YNAB's main highlight is enabling flexibility as expenses change, budgeting for non-monthly expenses and saving for the future. User's can adjust, use or "roll over" funds and focuses on money currently owned not future income, i.e. users can only spend what's available in that category and not anymore.

Key features:

- Zero-based budgeting (give every pound a job).
- Manual transaction entry through smartphone widgets/shortcuts and standard categories.
- Bank accounts can be imported (like other apps) and uses providers such as Plaid and MX.
- Shared budgets for families or couples to enable budgeting transparency.
- Real-time synchronisation across devices.

Operating environment:

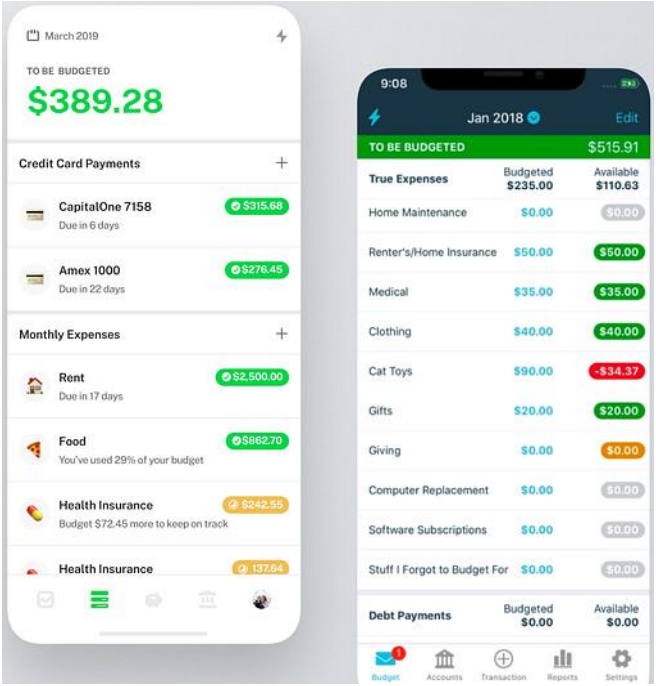
a. Platform:
   a. Android, iOS, web and Apple Watch
b. HW:
   a. A smartphone able to run modern apps or a computer that can run a website.
c. SW:
   a. Internet connectivity for bank account connection and file-based importing through CSV files.

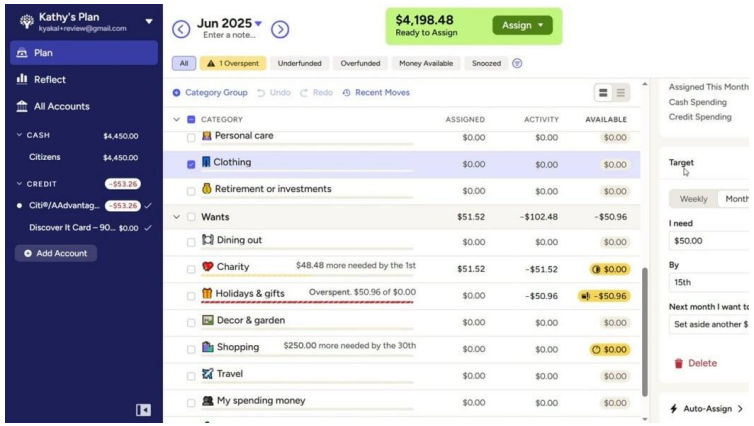
Pros:

a. "Giving every pound a job" enables proactive budgeting, rather than reflecting after money has been spent.
b. Flexibility when inputting data, users can manually add. transactions, optional direct imports and live, automated bank account syncing.
c. Allows users to move money between categories as priorities change.
d. Effective and unique methodology.
e. Wealth of resources and education workshops/videos to enable users to master the YNAB philosophy

Cons:

a. High subscription cost (£15/month), same cost as the highest Emma subscription
b. Steep learning curve - can be a complex methodology that isn't easy to pick up first time around although it can be powerful.
c. Direct bank imports are not instant and can take time - this increases user friction and may mean users turn to other apps.
d. Only retrospective transaction insights, nothing to tell you what impact spending something now will do.





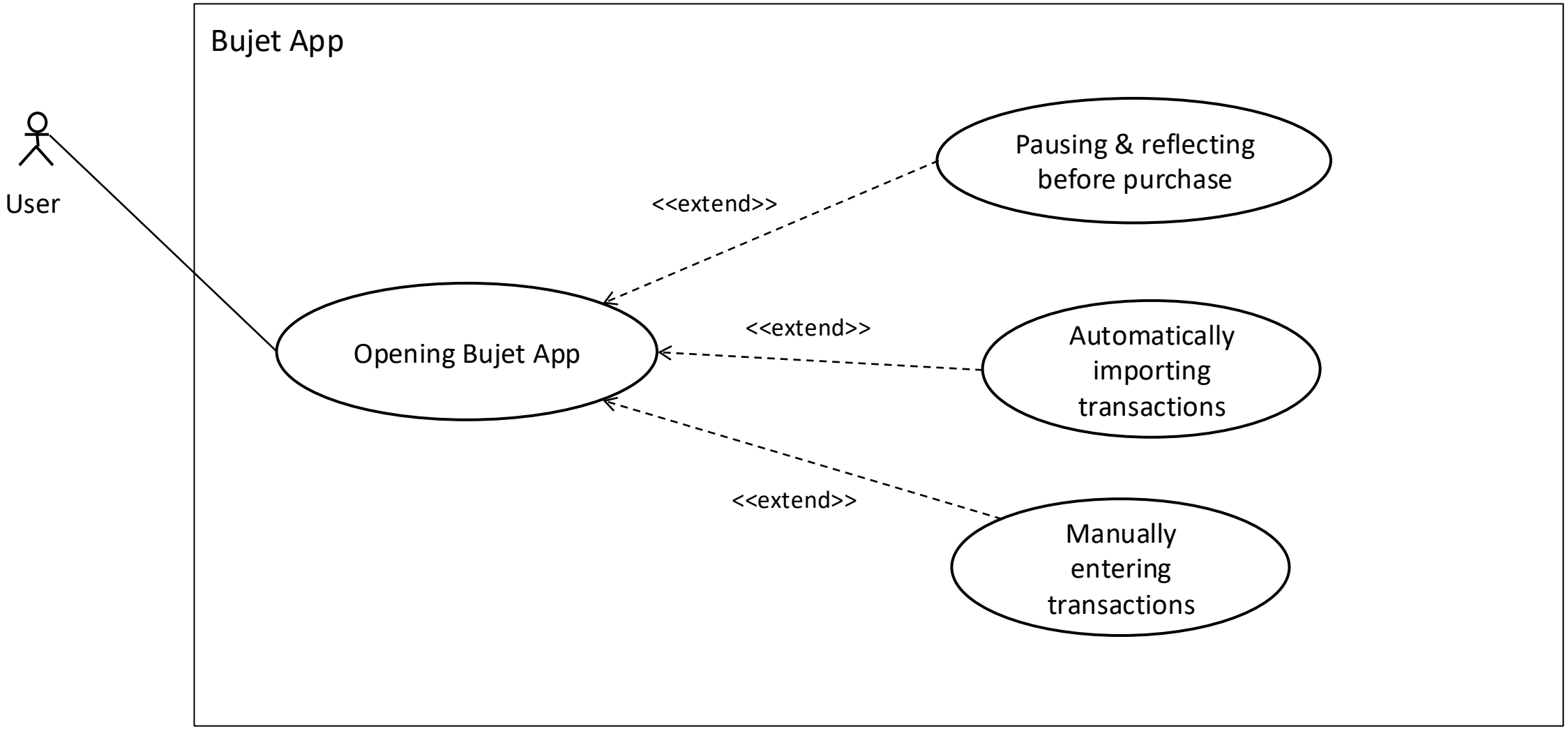
1. Project Ideation [20%] (cont'd)

1.3 Feature matrix of similar systems

Feature name	*Snoop*	*Emma*	*YNAB (paid)*	***Bujet***
Secure bank A/C connection	Yes	Yes	No	Yes
Manual transaction entry	Yes (paid)	Yes (paid)	Yes	Yes
Offline account	Yes (paid)	Yes (paid)	Yes	Yes
Pre-spend intervention	No	No	No	Yes (free)
"Money saved by pausing" metric	No	No	No	Yes (free)
Impulse spend notification & re-decision loop	No	No	No	Yes (free)

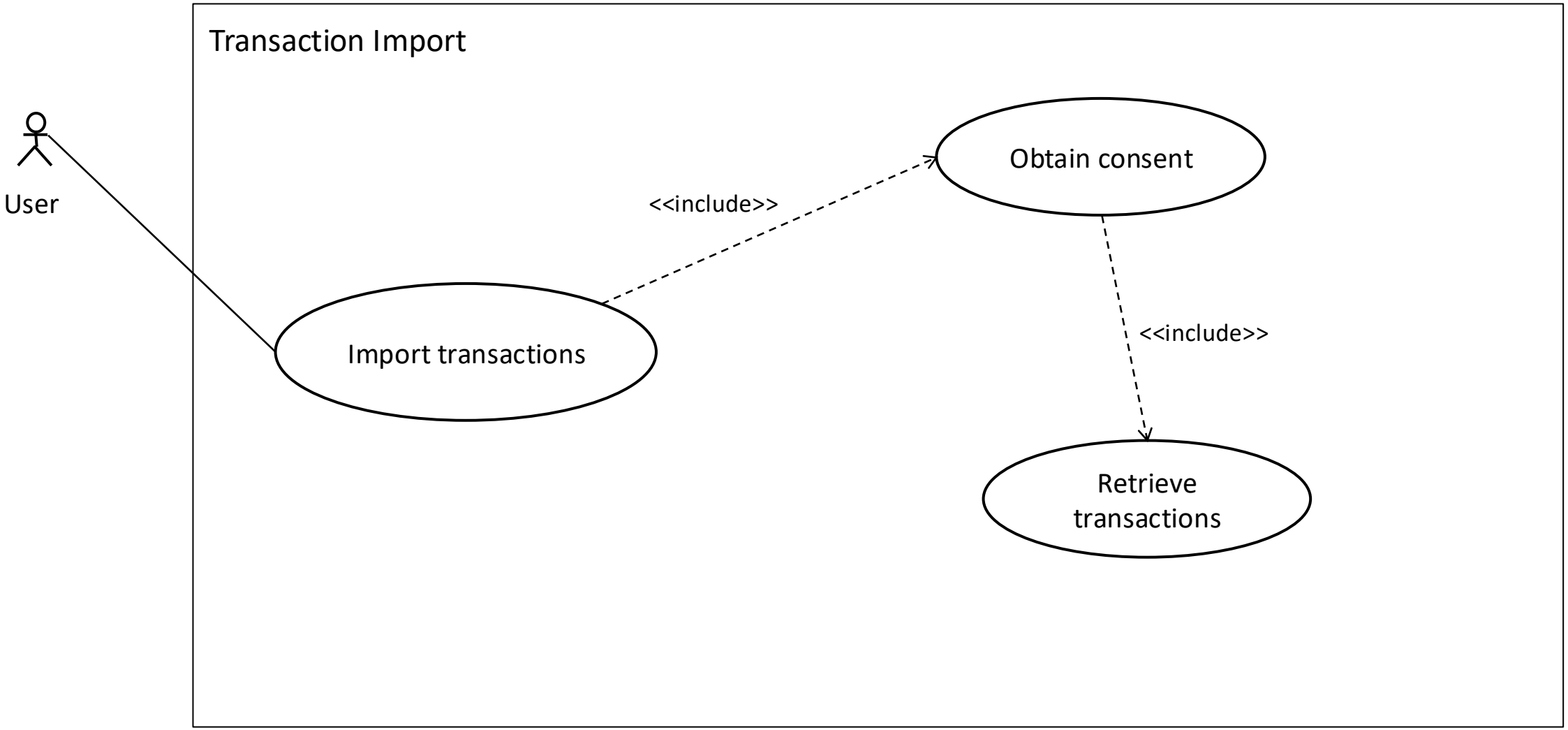
2. Requirements Analysis [25%]

2.1 Use Case Diagram



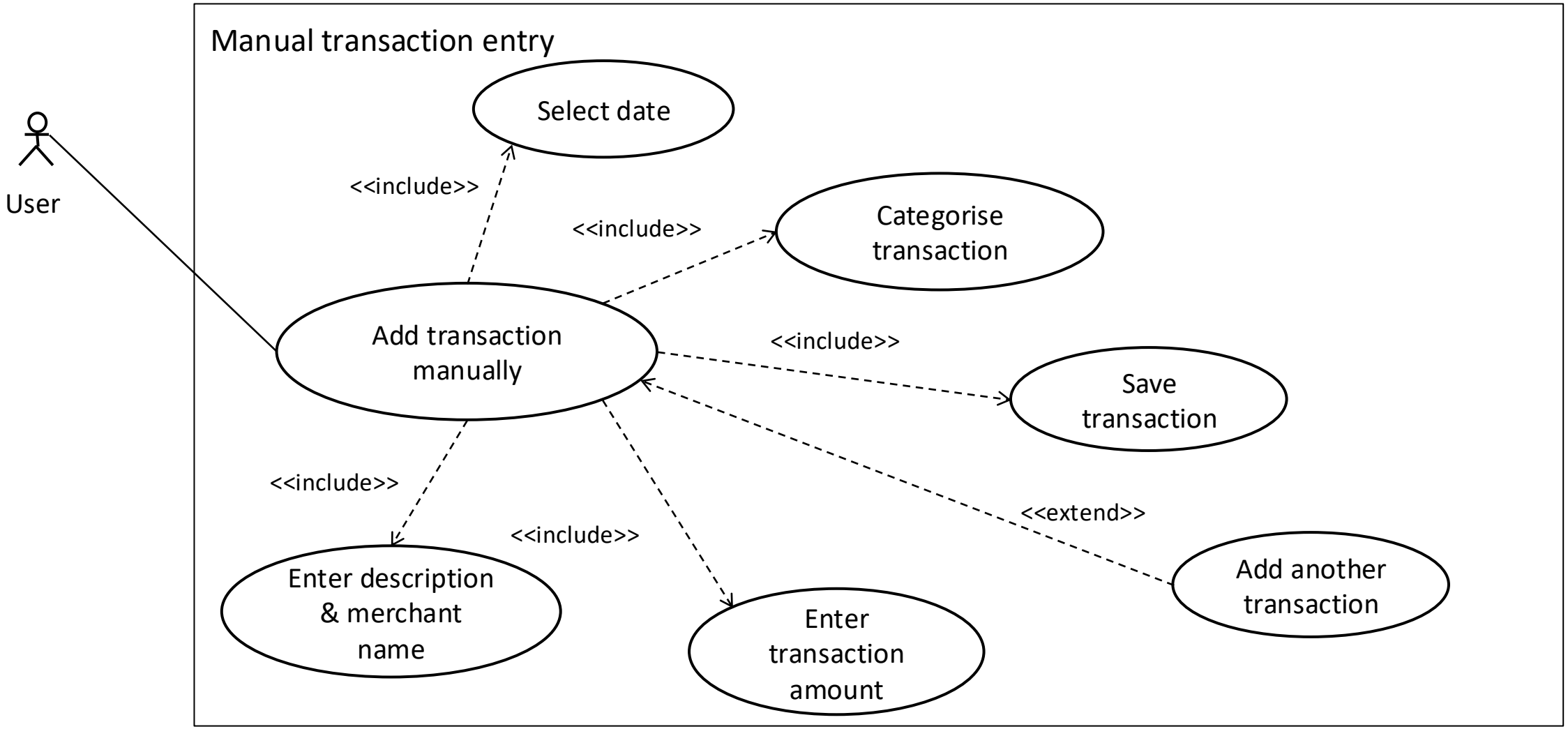
2. Requirements Analysis [25%]

2.1 Use Case Diagram



2. Requirements Analysis [25%]

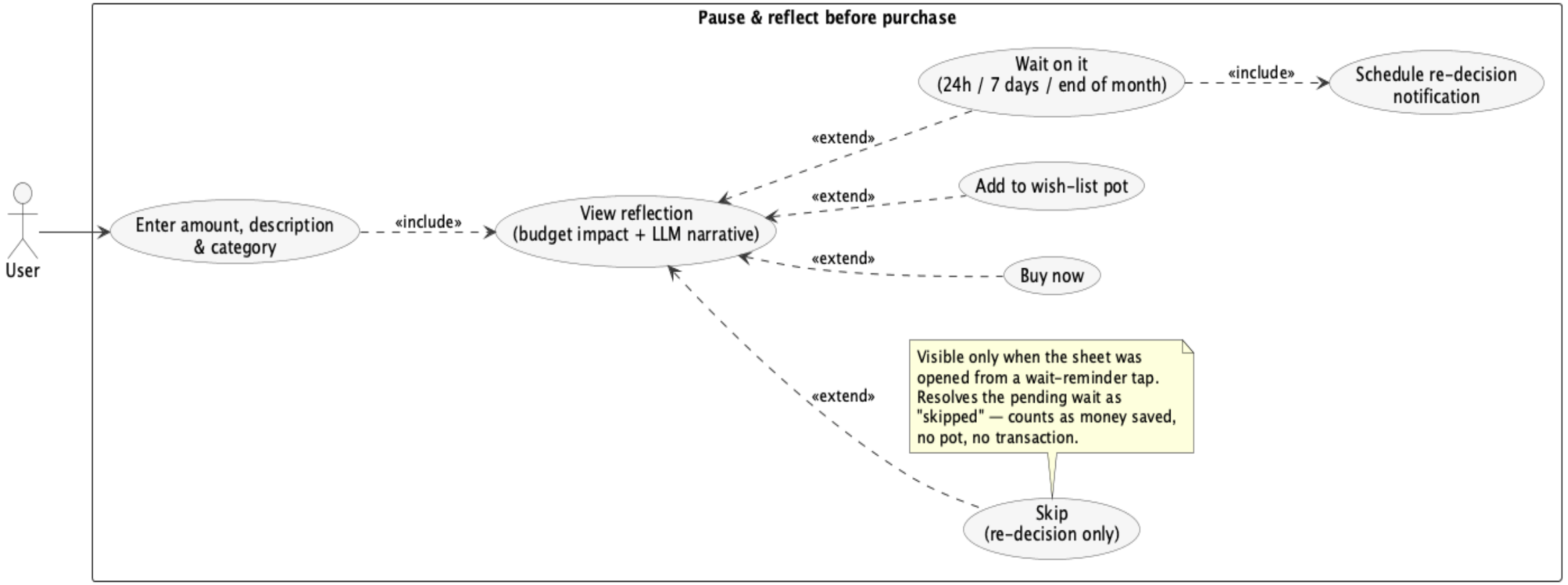
2.1 Use Case Diagram



2. Requirements Analysis [25%]

2.1 Use Case Diagram

Pause before purchase



2. Requirements Analysis [25%] (cont'd)

2.2 Use Case Table

Use case name:	Connect to user's bank account
Primary actor:	User
Secondary actors:	Truelayer API and bank itself
Summary:	User can link to their bank account through a secure API and receive automated transaction data to populate the Bujet app
Preconditions:	User opens app, user selects "import transaction data" picker and then "Connect bank account" button and internet connection is available. Truelayer configuration and backend services exist
Main success scenario:	1. User taps "connect bank account" button 2. Truelayer connection journey is opened 3. User grants consent 4. user completes bank authentication 5. Backend exchanges authentication code for an access token 6. System retrieves user transactions ready to display 7. App shows 'connected' status banner.
Alternatives:	7a User re-establishes connection to bank account after connection expired and starts connection flow again. 6a User only retrieves a particular date range when connecting (i.e. past 15 days or 30 days).
Exceptions:	3a User consent is denied → "Not connected" is shown in app. 4a Bank authentication fails/cancelled; return to app and show cause of error. 5a Backend token exchange fails, return error and display support options in app. 6b Transaction retrieval fails; shows "Connected but data pending" and retries. 6c Transaction retrieval fails completely; shows "Connected but data not imported, tap to try again".
Postconditions:	Bank connection created and backend receives access token. Initial transactions are received, stored and displayed to the user.

2. Requirements Analysis [25%] (cont'd)

2.2 Use Case Tables (min of 3 use case tables / person)

Use case name:	Manually record transaction
Primary actor:	User
Secondary actors:	None
Summary:	User can manually input transactions for offline accounts or when bank sync fails.
Preconditions:	User has at least one offline or connected account and app is unlocked through biometric authentication and usable.
Main success scenario:	1. User taps 'Add transaction' 2. User enters amount, date, description/merchant and selects associated account 3. User categorises transaction manually 4. App confirms successful transaction entry and updates totals.
Alternatives:	2a Quick-add: User enters only amount and merchant, system uses "today" as default and applies common merchant categories to auto categorise the transaction. 3a Create new category if required category is not found, user then returns to step 3 <<extend>>.
Exceptions:	2b User enters invalid input (negative or missing values); show invalid form and remain on form. 4a Save fails (storage issue), transaction not stored, show error and resolution protocol. 3b Auto categorisation conflict (two categories point to the same merchant, user selects category they want and revises rules).
Postconditions:	Transaction is saved; transaction has a category (if assigned) and budgets/analytics update.

2. Requirements Analysis [25%] (cont'd)

2.2 Use Case Tables (min of 3 use case tables / person)

Use case name:	Pause & review before purchase
Primary actor:	User
Secondary actors:	Calculated amount over or under budget.
Summary:	User enter a future transaction they want to buy and Bujet advises user about whether the transaction would put them over or under budget and the best option moving forward: buying, stalling or setting as a wish-list.
Preconditions:	App is open and app contains some transaction data (manual or imported) for the past month and budgets have been set.
Main success scenario:	1. User selects "I'm about to spend" button 2. System displays sheet where user can enter amount, what is it and the category it falls under 3. Reflect sheet appears, displaying amount transaction will take-up of the budget 4. User selects 'Buy now' and transaction is added.
Alternatives:	Alternatives: 4(alt): User adds it as a wish-list pot. 4(alt) User decides to wait on it 4(alt): On a wait-reminder re-decision the sheet shows an additional Skip button which resolves the pending wait as 'skipped' without creating a pot or transaction.
Exceptions:	3(exp): Apple Foundation LLM model doesn't load, app resorts to hard coded strings but same core data is displayed.
Postconditions:	Depending on what option is selected: Added as a manual transaction, notification is scheduled or wish-list pot is created

2. Requirements Analysis [25%] (cont'd)

2.3 Shall-statements

2.3.1 **First** sprint features (must include min of 3 NFRs)

S1.1: **A new user shall be able to connect to their bank account (FR).**
Description: A user can successfully navigate the connection journey of connecting their bank account and importing all transaction data, allowing further development for visualisation and transaction insight.

S1.2: **The system shall be able to retrieve user transactions from a connected bank account (FR).**
Description: After successful connection, the system shall be able to fetch and store all transactions for future use in charts, insights.

S1.3: **The app shall have iOS platform conventions and accessible touch interaction (NFR).**
Description: The app shall maintain consistent navigation patterns and iOS UI/UX conventions (taken from Apple's Human Interface Guidelines, found here) and provide touch targets of a minimum of 44x44pt to reduce mis-taps and improve accessibility.

S1.4: **The app shall remain responsive to user input (NFR).**
Description: The app shall meet the Apple responsiveness guidelines, by keeping discrete input response work under 100ms and ensuring frame updates stay with display refresh limits, preventing 'hangs' and 'hitches.

S1.5: **The app shall route TrueLayer authentication and data API requests through a backend service (NFR)**
Description: A small local server should be running on the developer's computer to simulate a backend server that deals with exchanging access tokens, authentication codes and calling the TrueLayer API. This is to increase security and make sure there are

2. Requirements Analysis [25%] (cont'd)

2.3 Shall-statements: 2.3.2 Test cases for **first** sprint

Test ID	Feature ID and Brief Description	Input Operations (Sequence and data)	Expected Output (text and/or drawing of the outputs)
T1.1	S1.1: Connect bank account with successful connection.	1) Launch app on iOS simulator. 2) Tap "Connect bank account" button. 3) Choose mock bank provider. 4) Consent approval screen. 5) Complete authentication and mock bank account connection. 6) Return to app home screen.	App shows "Connected" state on home screen. User is returned to app home screen without errors. Backend service receives auth code and completes exchange for access token (visible in backend with console logs).
T1.2	S1.1 Connect bank account, user cancels/denies consent (negative test).	1) Launch app. 2) Tap Connect bank account. 3) On consent screen , Cancel / Deny is tapped.	App shows "Not Connected" state. User sees clear message ("Connection cancelled" / "Consent required"). No account connection is saved (verified in logs).
T1.3	S1.2 Retrieve transactions with a data fetch after connection.	*Precondition:* complete T1.1. 1) Return to app home screen (triggers auto-refresh). 2) Request transactions for user selected range 3) Navigate to transaction screen.	Transaction list shows > 0 transactions (or expected mock count). Data persists after app restart (still visible). Backend logs show Data API calls were made for accounts/transactions.
T1.4	S1.2 Retrieve transactions in case of API/network failure handling (negative test).	*Precondition:* Connected to bank A/C. 1) Stop backend. 2) Launch app to home screen 3) Open transaction screen (auto-refresh)	App does not crash. App shows error state ("Unable to sync, try again"). UI remains stable and usable.
T1.5	S1.3 iOS conventions + touch targets (44×44pt) by doing an accessibility inspection.	1) Navigate to key screens: Home, Connections, Transactions, Review spending. 2) In Xcode, use Debug View Hierarchy. 3) Check primary interactive controls (connect button, add, back, filter) meet minimum size.	All primary tappable UI elements are ≥ 44×44pt (or have equivalent tappable padding). Navigation uses consistent iOS patterns (e.g., tab bar / navigation bar / back). No "tiny tap targets" on core flows.
T1.6	S1.4 Responsiveness, no hangs/hitches during key interactions.	1) Launch app. 2) Perform 10 quick taps through common flow: Home → Connections screen → Back → Transactions → Review spending. 3) Scroll transaction list rapidly for 10 seconds. Can be verified using Instruments "Time Profiler" or Xcode performance tools.	No noticeable UI freezes. Taps trigger visible feedback quickly. Scrolling remains smooth (no repeated stutters). Can be verified using Xcode profiling.
T1.7	S1.5 Backend routing ensures there are no direct TrueLayer calls from the iOS app.	1) Start backend server locally (note port). 2) Run app and complete T1.1 + T1.3. 3) Observe network traffic using backend logs.	All requests from iOS app go to backend, not to TrueLayer domains directly. Backend logs show performs token exchange + Data API calls. This demonstrates "no frontend calls".
T1.8	S1.5 Backend unavailable means the app fails safely.	1) Ensure backend is stopped. 2) Open app. 3) Tap Connect bank account or Tap "Refresh" button on transaction screen.	App shows a clear error ("Backend unavailable") and suggests next action (start server / retry). App remains usable for non-network screens (e.g., UI still navigable). No crash.

2. Requirements Analysis [25%] (cont'd)

2.3 Shall-statements

2.3.3 **Second** sprint features (must include min of 3 NFRs)

S2.1: **Users shall be able to manually add transactions (FR).**
Description: A user should be able to manually add custom transactions, including categorisation, to offline and online (live bank account connection) app experiences. This shall include date, merchant name, amount and associated account.

S2.2: **The app shall validate transaction data before saving (FR).**
Description: The system shall validate manual transaction input to maintain data quality. This includes variables such as amount being non-zero, date must be valid and within a reasonable time frame, account must exist and category must exist.

S2.3: **The app shall store financial data securely on-device using AES-256 (NFR).**
Description: financial data stored locally (manual transactions, categories, offline accounts) shall be protected using iOS security mechanisms, such as using Keychain to store secrets/tokens and using iOS Data Protection class (encryption at rest) to store files.

S2.4: **The app shall prevent loss of manually entered transaction data (persistence across restarts) (NFR).**
Description: Manually entered transactions shall persist across app restarts and device reboots. The app shall also prevent accidental loss of an in-progress transaction by warning the user before discarding unsaved inputs (e.g., leaving the screen with unsaved changes).

S2.5: **Users shall be able to edit or delete manually entered transactions without internet connection (NFR).**
Description: Users shall be able to create, edit, and view manually entered transactions without an internet connection. Any sync-related operations (if present) shall occur only when connectivity returns, without blocking local entry.

2. Requirements Analysis [25%] (cont'd)

2.3 Shall-statements 2.3.4 Test cases for **second** sprint

Test ID	Feature ID and Brief Description	Input Operations (Sequence and data)	Expected Output (text and/or drawing of the outputs)
T2.1	S2.1 Manual add transaction results in successful save.	1) Launch app. 2) Navigate to Transactions screen. 3) Tap Add Transaction. 4) Enter: Date = today, Merchant = "Coffee Shop", Amount = 3.50, Account = "Cash/Offline", Category = "Food & Drink". 5) Tap Done.	Transaction is created and appears in transaction list with correct details. Category totals & spending tab update. No error message shown.
T2.2	S2.1 Manual add transaction and appears in insights/summary.	1) Complete T2.1. 2) Navigate to "Review spending. 3) Confirm transaction appears.	Spending summary reflects new manual transaction (e.g., "Food & Drink" increases by £3.50). Transaction appears in transaction list.
T2.3	S2.2 Validation amounts must be non-zero (negative test).	1) Tap Add Transaction. 2) Enter Amount = 0.00 (or leave amount blank), fill other fields normally. 3) Tap Done.	App prevents saving. Clear alert message shown (e.g., "Amount must be greater than 0"). User remains on form; no transaction is created.
T2.4	S2.2 Validation of date must be valid/reasonable (negative test).	1) Tap Add Transaction. 2) Set Date far in the future (e.g., +2 years), fill other fields. 3) Tap Save.	App prevents saving. Alert message shown (e.g., "Date must be within a reasonable range"). No transaction is created.
T2.5	S2.2 Validation of account & category must exist (negative test).	1) Tap Add Transaction. 2) Leave Account unselected OR attempt to save without category selected. 3) Tap Save.	App prevents saving. Inline errors shown near missing fields (e.g., "Select an account", "Select a category"). No transaction is created.
T2.7	S2.3 Secure storage of financial data (encrypted at rest).	1) Locate where manual transactions are stored (e.g., app database/file). 2) Inspect file protection setting (project entitlements or file attributes). 3) Verify stored financial files use an iOS Data Protection class.	Evidence shows stored financial files are encrypted on disk and protected by an iOS file protection class.
T2.8	S2.4 Persist manually entered data after app restart.	1) Create a manual transaction (use T2.1). 2) Force-close app. 3) Reopen app and return to Transactions screen.	Previously created manual transaction is still present (no data loss). This demonstrates persistence across app restarts.
T2.9	S2.4 Prevent accidental loss of in-progress transaction (unsaved-changes warning).	1) Tap Add Transaction. 2) Enter Merchant and Amount but do not tap Save. 3) Attempt to go back / swipe to dismiss / navigate away. 4) Choose "Cancel" or "Keep Editing" when prompted.	App shows a warning prompt (e.g., "Discard changes?"). If user chooses "Keep Editing", the form retains entered values; nothing is lost unless user explicitly discards.

2. Requirements Analysis [25%] (cont'd)

2.3 Shall-statements

2.3.5 **Third** sprint features (must include min of 3 NFRs)

S3.1: **The user shall see a pre-spend reflection sheet (FR).**
Description: The app shall present a "Pause & Reflect" sheet that asks for transaction amount, a brief description and the category it falls under. A "reflect" button appears underneath, which directs the user to a decision sheet.

S3.2: **The user shall be able to set reminders in the decision sheet which contain a deep-link re-decision flow (FR).**
Description: The "Wait on it" button asks the user when they would like a reminder notification sent (24 hours, 7 days or 1 month), which pre-populates the Pause & Reflect flow and automatically directs to a re-decision flow.

S3.3: **The user shall be able to add transaction to a wish-list pot or skip action upon re-decision flow (FR).**
Description: On the first decision flow, the user can choose to make the transaction a wish-list pot so they can save towards it and buy it in the future. On the re-decision flow, the user can decide to skip the transaction entirely, which is added to the "Saved by pausing this month" banner reinforcing the savings by reflecting before you buy.

S3.4: **The app shall save the "Money saved" amount as a single record, enhancing data integrity (NFR).**
Description: Each "Wait on it" decision creates exactly one pending record, keyed by the original proposal ID. When the notification fires and the user re-decides, the existing record is resolved rather than a new one being created. So, the metric stays honest under repeated wait/snooze cycles.

S3.5: **The app shall use the iOS on-device LLM used for coach narration for maximum privacy (NFR).**
Description: This on-device LLM personalises the text, meaning the user is more likely to think twice about what they're buying. Having it on device means the user's transactional data isn't sent to an external server and reduces text generation latency.

S3.6: **The notification for transaction reminder shall pre-fill reflection sheet < 100ms after app start (NFR)**
Description: Keeping pre-fill time to a minimum reduces user friction meaning they're more likely to interact with the re-decision sheet and give an honest answer. This comes from Apple's responsiveness guidelines, which state any input should be 100ms or less.

2. Requirements Analysis [25%] (cont'd)

2.3 Shall-statements: 2.3.6 Test cases for **third** sprint

Test ID	Feature ID and Brief Description	Input Operations (Sequence and data)	Expected Output (text and/or drawing of the outputs)
T3.1	S3.1 Pause &Reflect sheet opens and accepts a valid proposal.	1) Launch app. 2) Tap "I'm about to spend…". 3) Enter Amount = 35, Description = Wool jumper, Category = Shopping. 4) Tap Reflect.	Decision view appears, showing budget impact for Shopping, LLM-generated narrative paragraph, four-button choice (Buy now, Wait on it, Add to Pot, no Skip on first decision). No Skip button visible (it's re-decision only).
T3.2	S3.1 Validation – Reflect is disabled for zero/empty input (negative test).	1) Tap "I'm about to spend…". 2) Leave amount blank (or 0) and/or description blank. 3) Observe the Reflect button state.	Reflect button is disabled. If user forces submission, inline error appears ("Enter a valid amount." / "Add a short description.") and no decision view is shown.
T3.3	S3.1 No-budget nudge fires when the chosen category has no limit set.	1) Ensure no budget is set for Shopping. 2) Open the sheet, enter £35/"Wool jumper"/Shopping. 3) Tap Reflect.	"Set a budget?" alert appears with two options: Set a budget (presents budgets sheet on top of the flow) and continue without budget (proceeds to evaluation). Input is preserved across the alert.
T3.4	S3.2 "Wait on it" schedules a local notification with the proposal encoded in userInfo.	1) Complete T3.1 to land on the decision view. 2) Tap Wait on it →choose 24 hours. 3) Inspect pending notifications or wait for fire (fires in ~5 s).	Notification fires and contains amount, item description and category. Title reads "Still want Wool jumper?".
T3.5	S3.2 Tapping the wait reminder notification deep-links into a pre-filled Pause & Reflect sheet sitting on the decision view.	1) Complete T3.4 and background the app. 2) Wait for the banner to fire (~5s). 3) Tap the banner.	App foregrounds; reflect sheet opens automatically; amount field shows 35, description shows Wool jumper, category shows Shopping; the user lands directly on the decision view (input step is skipped); a Skip button is now visible alongside Buy now/Wait/Add to Pot.
T3.6	S3.3 "Add to Pot" on the re-decision flow resolves the pending wait as skipped and creates a wish-list pot (does not write a duplicate log).	1) From the deep-linked re-decision sheet (T3.5), tap Add to wish-list pot.	A new Goal is created with target amount = 35. The original pending intervention for that proposal moves to outcome = skipped and increases "Saved by pausing this month" by £35.

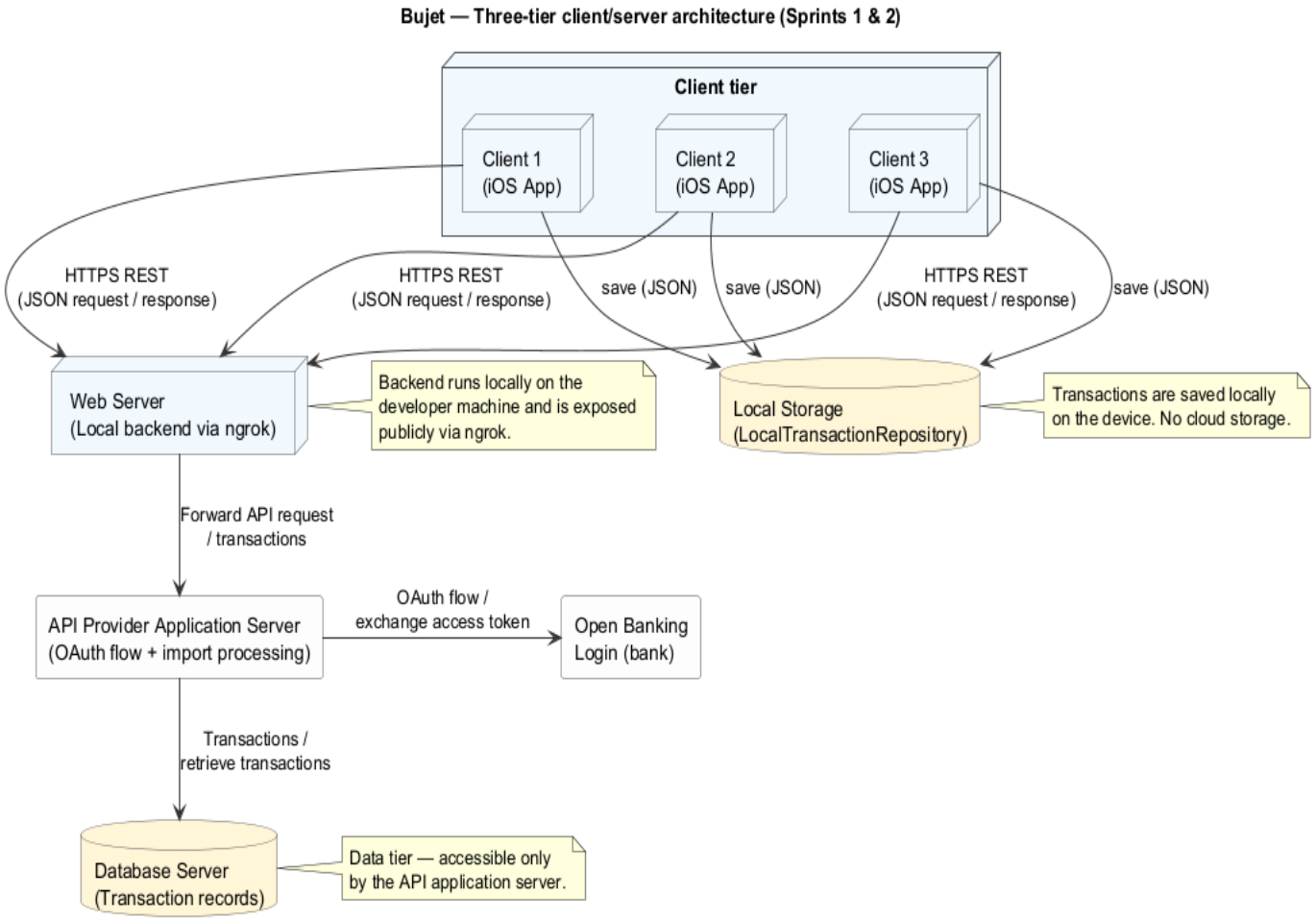
2. Requirements Analysis [25%] (cont'd)

2.3 Shall-statements: 2.3.6 Test cases for **third** sprint (cont)

Test ID	Feature ID and Brief Description	Input Operations (Sequence and data)	Expected Output (text and/or drawing of the outputs)
T3.7	S3.3 "Skip" on the re-decision flow resolves the pending wait as .skipped with no pot and no transaction.	1) From the deep-linked re-decision sheet (T3.5), tap Skip.	Sheet dismisses. The pending intervention is now .skipped. No Goal exists for the item; no Transaction is written. "Saved by pausing this month" increases by £35.
T3.8	S3.4 Repeated snoozing does not double-count toward "saved by pausing" (single-record data integrity NFR).	1) Complete T3.4 → T3.5 (1st re-decision). 2) On the re-decision sheet, tap Wait on it → 24 h. 3) When the banner fires again, tap it. 4) Tap Skip.	Across the whole sequence, exactly one intervention exists for the proposal. Its final state is skipped. "Saved by pausing this month" reflects £35, not £70.
T3.10	S3.6 Notification → pre-fill latency under 100ms (Apple HIG responsiveness).	1) From a cold-start state, fire the wait banner (~5 s after Wait) 2) Tap the banner. 3) Use Instruments → Time Profiler/signposts around reminder router and the intervention flow that follows.	Time from reminder router receiving the prefilled proposal to the decision view becoming visible is <100ms (excludes LLM evaluation, which runs after the sheet is on screen). No dropped frames during sheet presentation.

3. Architecture and Object-Oriented Design [25%]

3.1 External architecture diagram



Why this architecture?

- This architecture is the 'multi-tier client-server' architecture pattern, which protects against users reading and intercepting the actual API request from the app itself.
- Reusability: if the API request style changes, the app itself doesn't need to be updated only the web server.
- This also enables the web-server to re-direct traffic to another application server if one fails.

What are the disadvantages?

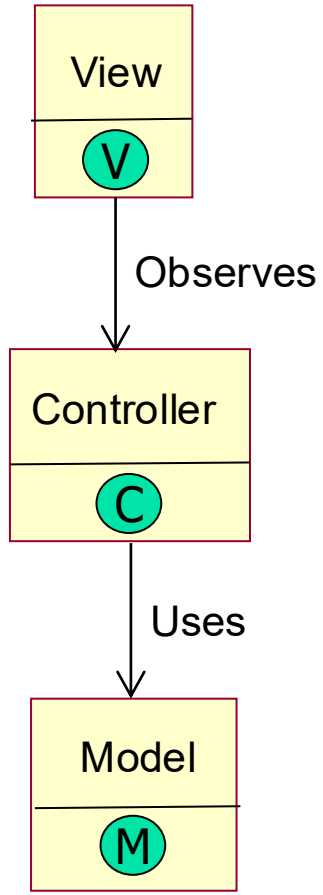
- However, maintaining this internal web server can be costly.
- Increases latency for each transaction request.

How does it work?

- Any client requests are passed through a local NodeJS server before being sent through an API request to the server that retrieves the bank transactions.

3. Architecture and Object-Oriented Design [25%]

3.1 Internal architecture diagram - MVC



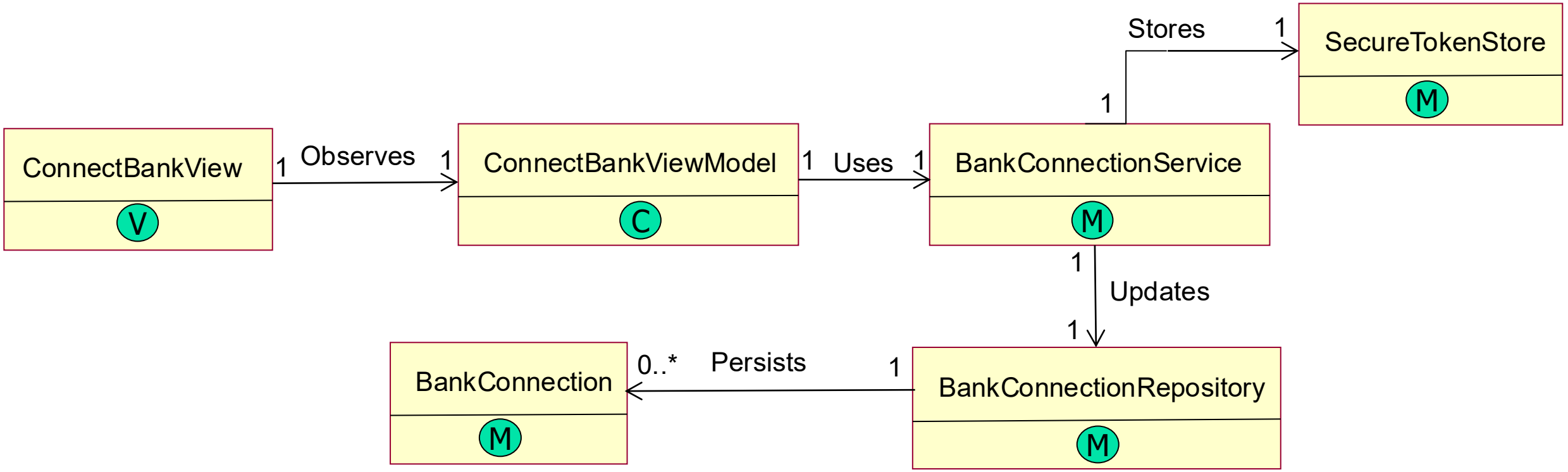
What is the internal architecture used in Bujet?
MVC (Model-View-Controller). The Model is concerned with data and business logic. It knows how to fetch, store, validate and manipulate data but knows nothing about how it's displayed. The View is the UI. It renders what the user sees and forwards user input (clicks, typing) to the controller. It only displays; it is unable to decide what to do. The Controller is the middleman between View and Model. It receives input from the View and tells the Model what to update and tells the View what to show (from the Model). It's the "brains" coordinating the other two.

Why was this architecture chosen? What are the benefits?
It increases separation of concern, so changes to one class do not ripple into the others. This reduces tight coupling between classes and means changes can be built incrementally. This also makes it easier to debug, with clear boundaries between each class. It increases reusability, e.g. multiple Views can share the same Model and enables models to be tested without the UI, which makes unit testing far easier.

What are the disadvantages? What's bad about MVC?
High complexity overhead, for small apps MVC is overkill and the functionality of each class could be reduced into one. Controllers can bloat, meaning they tend to absorb logic that doesn't clearly belong elsewhere, and this increases View-Controller coupling so separation of concern is lost and debugging is more difficult.

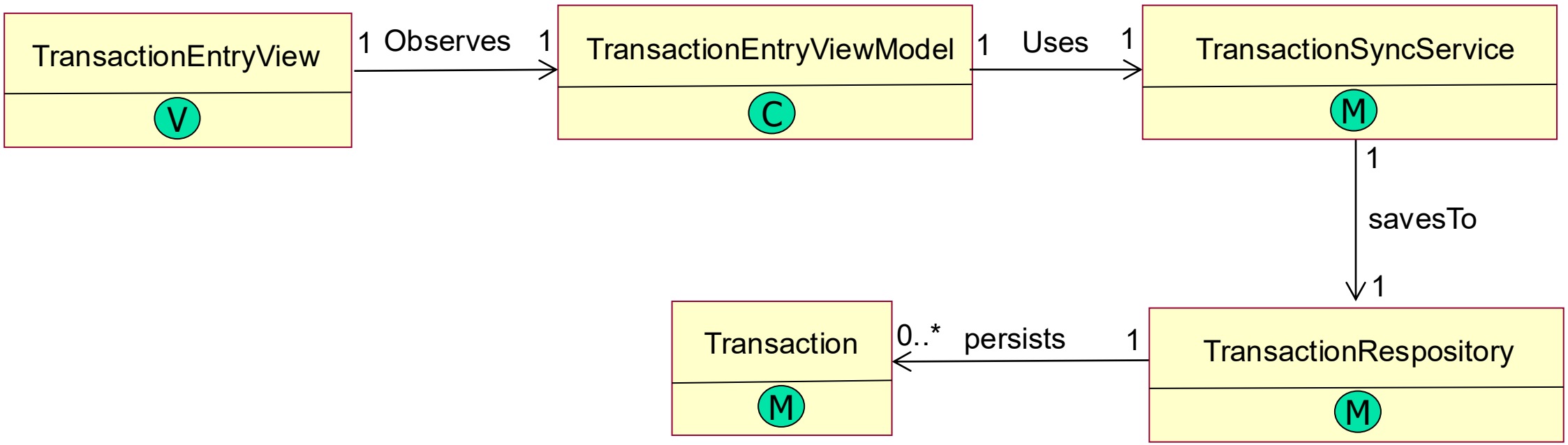
3. Architecture and Object-Oriented Design [25%] (cont'd)

3.2 Class diagram – Sprint 1



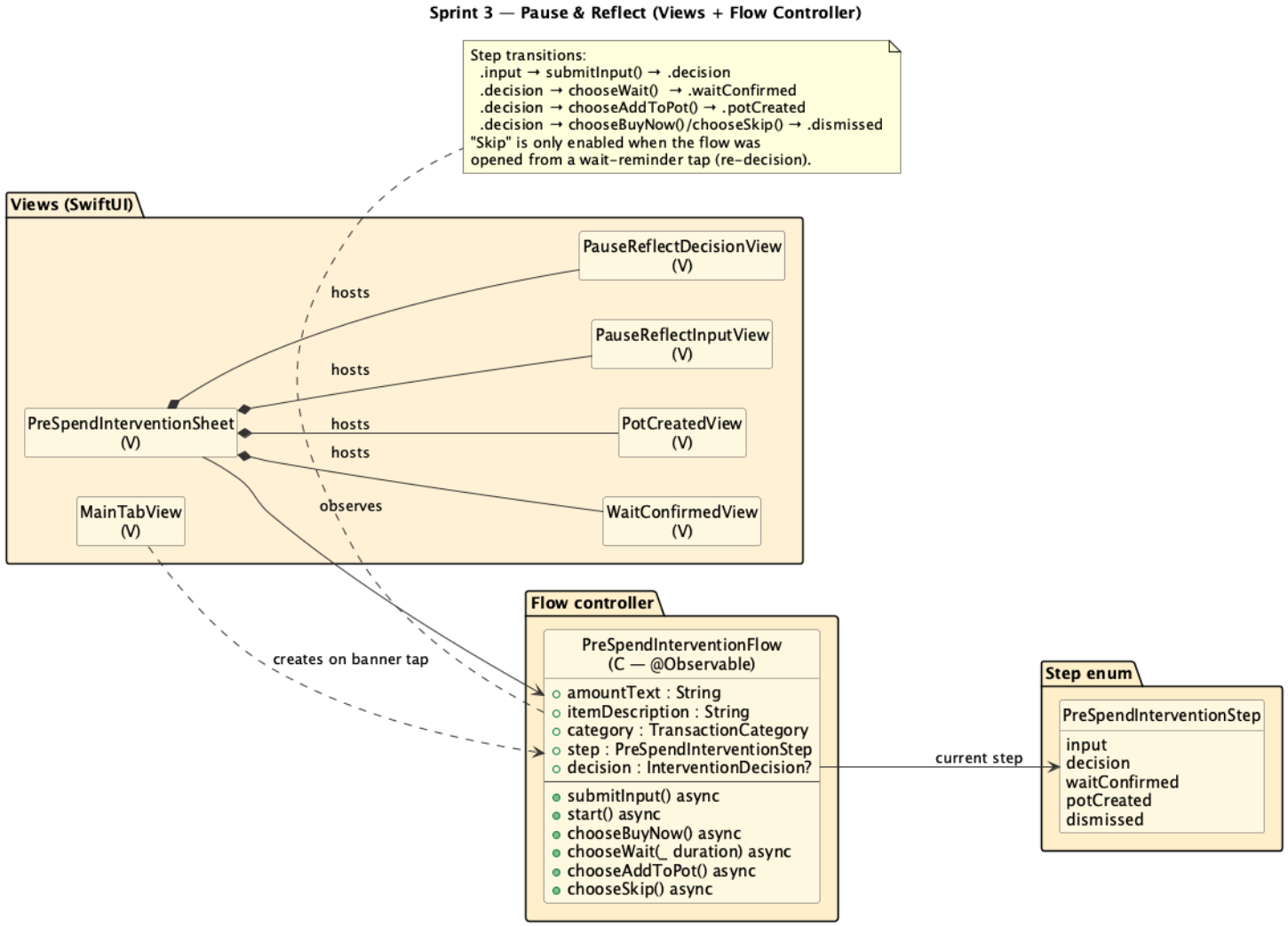
3. Architecture and Object-Oriented Design [25%] (cont'd)

3.2 Class diagram – Sprint 2



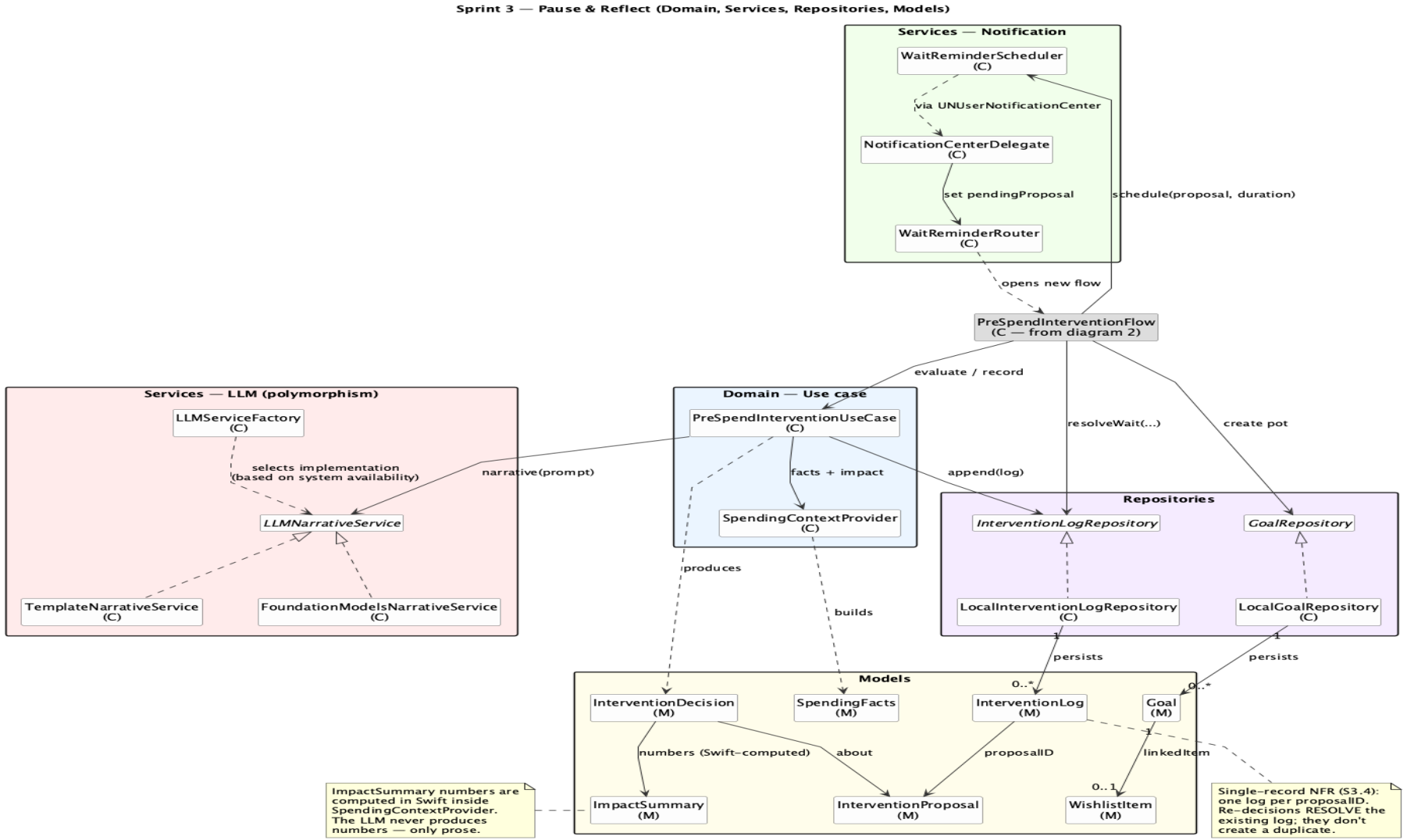
3. Architecture and Object-Oriented Design [25%] (cont'd)

3.2 Class diagram – Sprint 3 Views and Flow



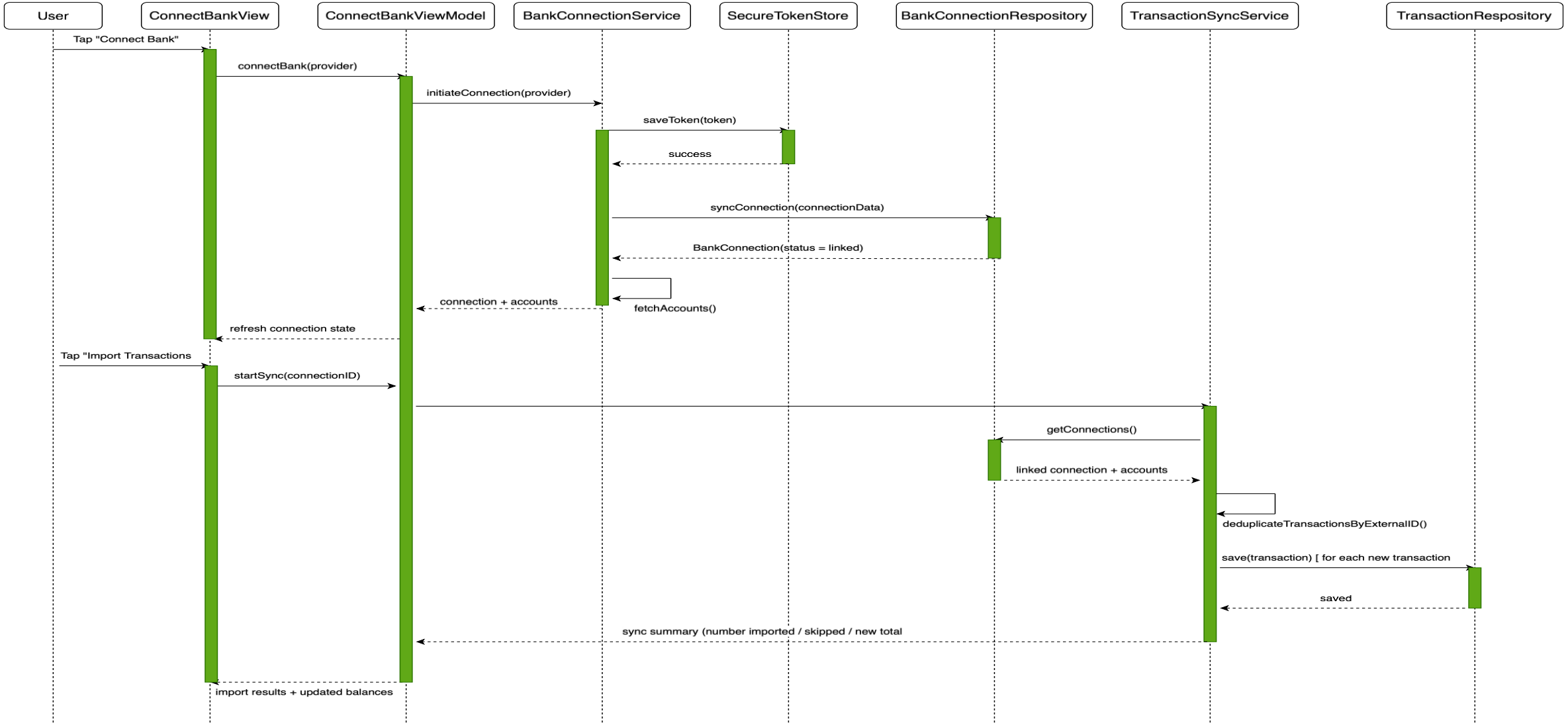
3. Architecture and Object-Oriented Design [25%] (cont'd)

3.2 Class diagram – Sprint 3 Data Flow



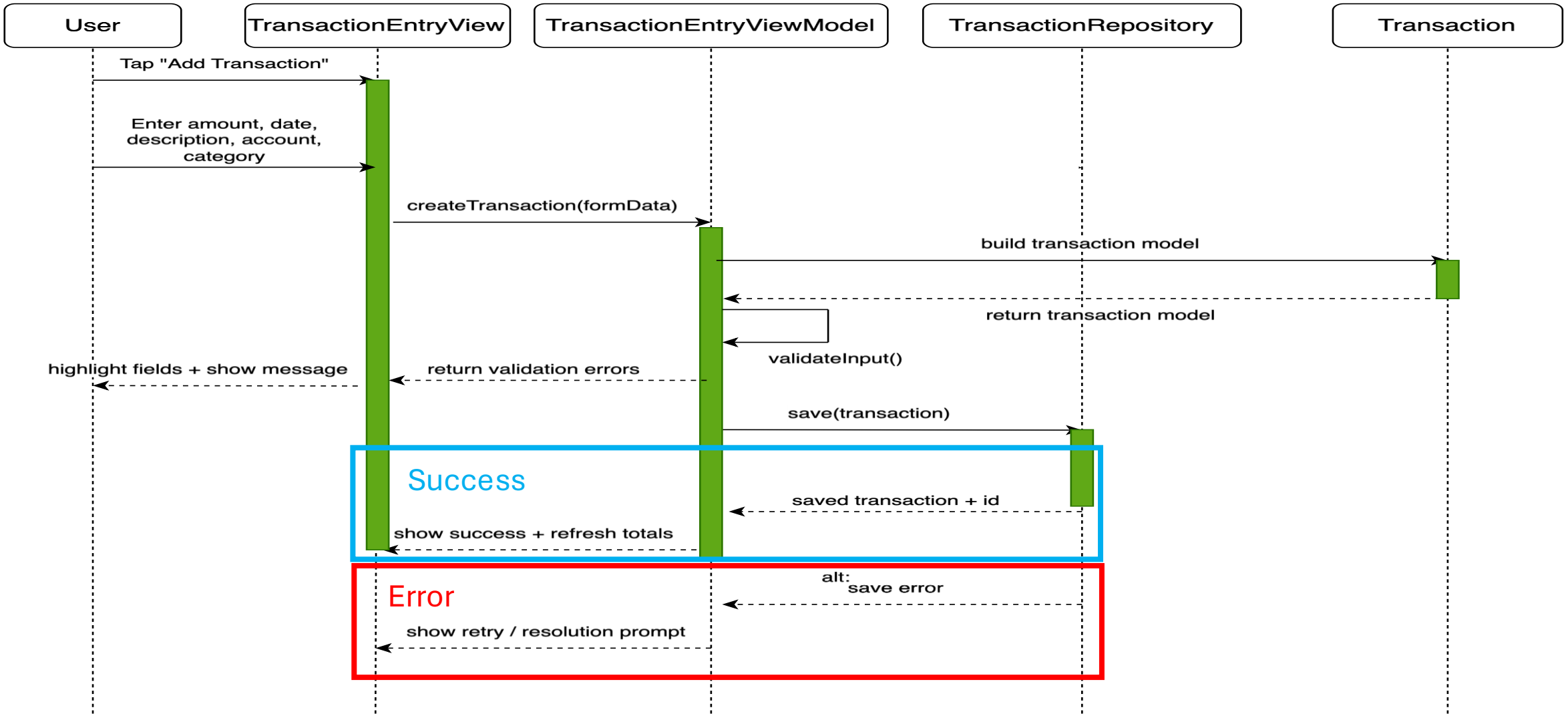
3. Architecture and Object-Oriented Design [25%] (cont'd)

3.3 Message sequence diagram – Connect to user's bank account



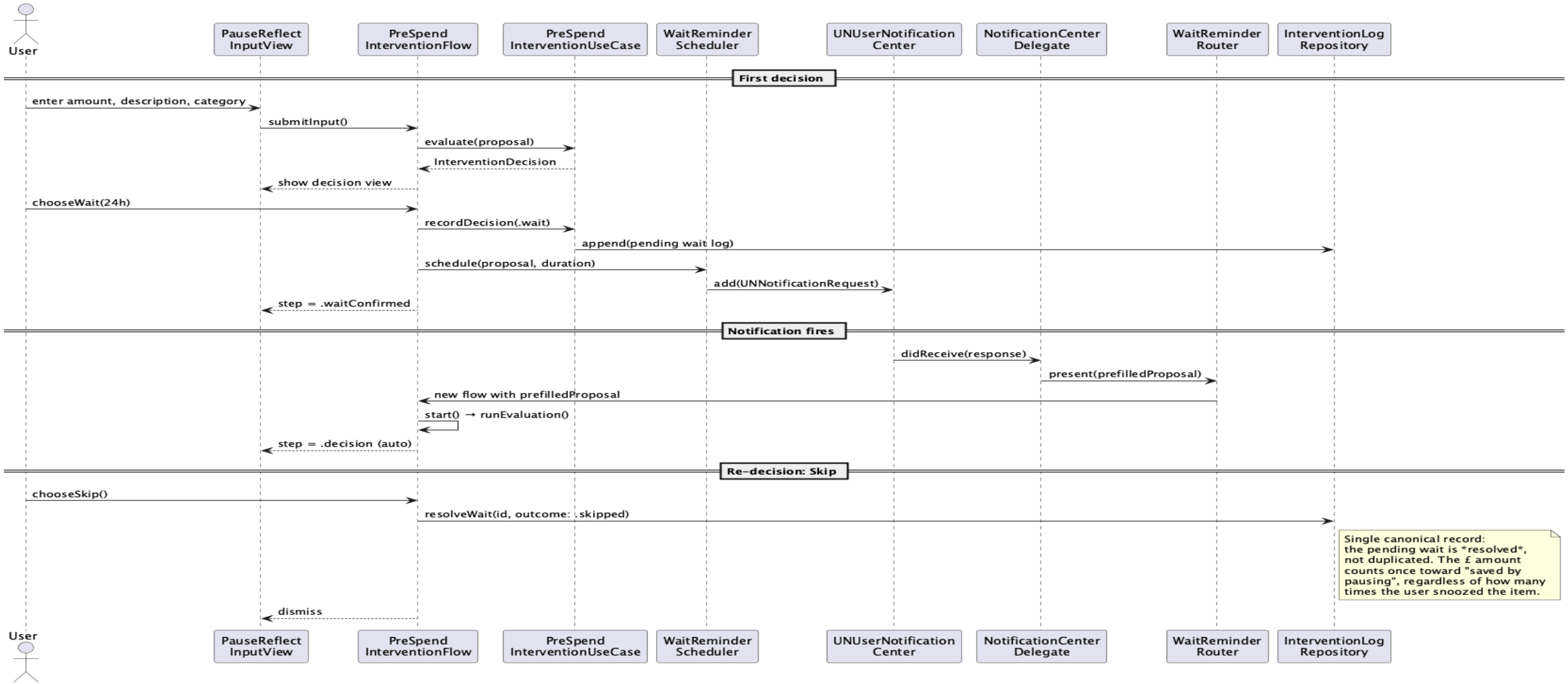
3. Architecture and Object-Oriented Design [25%] (cont'd)

3.3 Message sequence diagram – Manually record transaction



3. Architecture and Object-Oriented Design [25%] (cont'd)

3.3 Message sequence diagram – Pause & Reflect flow (snooze and re-decision)



3. Architecture and Object-Oriented Design [25%] (cont'd)

3.4 Detailed class description (min of three classes / person)

BankConnectionService

secureTokenStore: SecureTokenStore
bankConnectionRepository: BankConnectionRepository
linkedConnection: BankConnection?
linkedAccounts: [Account]
lastError: String?

initiateConnection(providerName: String) -> Void
// Starts the bank-linking flow for the selected provider.
fetchAccounts(connectionID: UUID) -> [Account]
// Retrieves accounts associated with the linked bank connection.
storeTokenSecurely(token: String) -> Void
// Delegates secure token storage to SecureTokenStore.
syncConnectionStatus(connectionID: UUID, status: String) -> Void
// Persists the current connection state through the repository layer.

LocalTransactionRepository

transactions: [Transaction]
storageMode: LocalPersistentStore
supportsImportedAndManual: Bool
lastSavedTransaction: Transaction?

save(transaction: Transaction) -> Void
// Stores a newly created manual or imported transaction.
fetch(accountID: UUID?) -> [Transaction]
// Returns stored transactions, optionally filtered by account.
update(transactionID: UUID, updatedTransaction: Transaction) -> Void
// Updates an existing transaction record.
delete(transactionID: UUID) -> Void
// Removes a transaction from persistent storage.

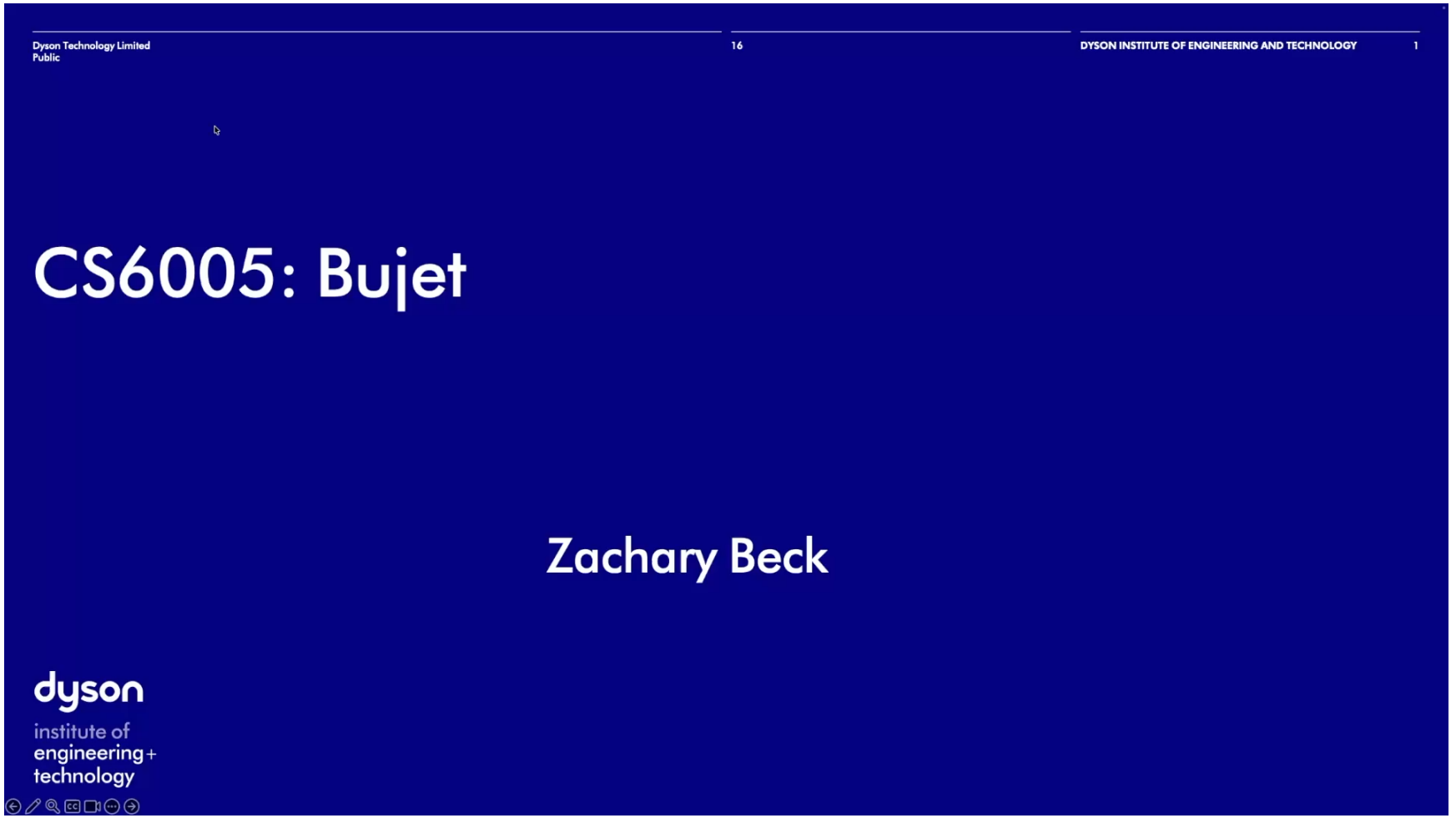
PreSpendInterventionFlow

useCase: PreSpendInterventionUseCase
goalRepository: GoalRepository
interventionLogRepository: InterventionLogRepository
waitReminderScheduler: WaitReminderScheduler
originalProposalID: UUID?
step: PreSpendInterventionStep
amountText: String
itemDescription: String
category: TransactionCategory
decision: InterventionDecision?
isEvaluating: Bool
isRecording: Bool

submitInput() async -> Void
// Validation
start() async -> Void
// Decision entry point
chooseBuyNow() async -> Void
// Appends impulse to transactions.
chooseWait(_ duration) async -> Void
// Schedules notification and duration
chooseAddToPot() async -> Void
// Creates wish-list goal from item, skipped on re-decision.
chooseSkip() async -> Void
// Only re-decision, powers "saved by pausing"

4. Evidence of Implementation [25%]

4.1 Video recording of the working system (max of 10 minutes long)



5. Reflection [5%]

Lessons learnt and what I would improve

- I learnt that software architecture is a very important step in the process of developing software from idea to product. It underpins the scalability of the software when it comes to adding extra features and improves readability from a developer maintenance perspective. If I had another chance, I'd make Figma designs first to visually understand app flow, data flow and nuances present in my idea. This would significantly improve my architecture diagram and overall system understanding before writing any code. I made the mistake of writing code too quickly and not thinking about design and architecture, which meant I had to backtrack and re-factor afterwards. I also learnt that even with advanced AI tools there is still significant human interaction, guidance and thought to make the product understandable and easy to use.

AI tool usage reflection

- I use M365 Copilot (basic licence) and Claude Opus 4.7 within Claude Code. M365 Copilot is good for quick questions and generating snippets of code to solve a specific problem. I found it struggled to understand the entire project context and is limited to uploading only 4-5 files, which can't be Swift, so I converted the files to text files but it still wasn't an ideal solution. It required a lot of manual oversight and human intervention. I was regularly creating new chats to reset the context window, because over a long conversation Copilot would lose focus and the code would start to break, but in doing so I'd have to re-explain the context in the next chat – slowing me down. Claude Opus 4.7 within Claude Code, on the other hand, was significantly better. It runs in the terminal within the folder of my project, so it has full context of my project and can also see all the previous git commit history, so it understands the entire project end-to-end.

- My usual approach was discussing the feature I wanted to implement and my rough idea of how it would work, pasting my architecture diagram, gathering feedback and agreeing on the best implementation. I also learnt to add skills to Claude so Claude had the most up-to-date information and wasn't using outdated APIs, which could throw deprecation warnings [1]. I also improved my prompting to get better, more accurate and less buggy results by reading the Claude API docs from Anthropic [2]. Then Claude would implement the feature, and I would tweak the visuals or change the flow if it didn't make sense. Usually this happens when Claude assumes certain behaviours or visuals, so I've learnt to think more about exact flow, error messages and ambiguity, so Claude reduces assumptions. I've also learnt, from using Claude, that it is a very powerful tool, but care must be taken to ensure conventions are adhered to and Claude reuses as much of the codebase as possible, otherwise an excessive number of new files can be created and this 'spaghettifies' the code, reducing maintainability and developer understanding. I learnt this conventional architecture from reading this article about iOS architecture conventions [3]. Overall, even if Claude writes the code, it is still imperative to understand the architecture and internalising the codebase. I took time to internalise the codebase after a feature was implemented. This helped me write the architecture diagrams and ensure everything in my portfolio made sense.

Team formation reflection

- I did my project individually because no one else had a similar system and only a few people were developing iOS apps. This made is difficult to team up and incorporate both features of our respective apps. I also preferred to have full ownership over my app, ensuring it was exactly as I wanted. As an individual it meant maximum autonomy and all the ideas in my app were my ideas, which reduced the amount of time having to communicate and align with another member of the team for a new feature implementation. However, being an individual also meant there wasn't a second pair of eyes to look over the code and app to suggest improvements and different